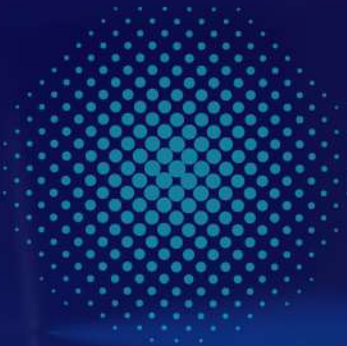
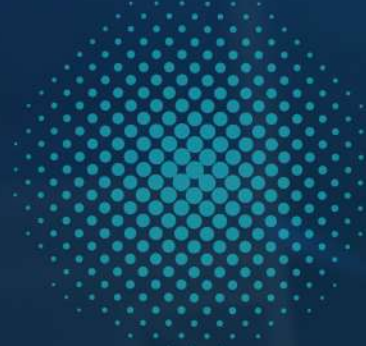




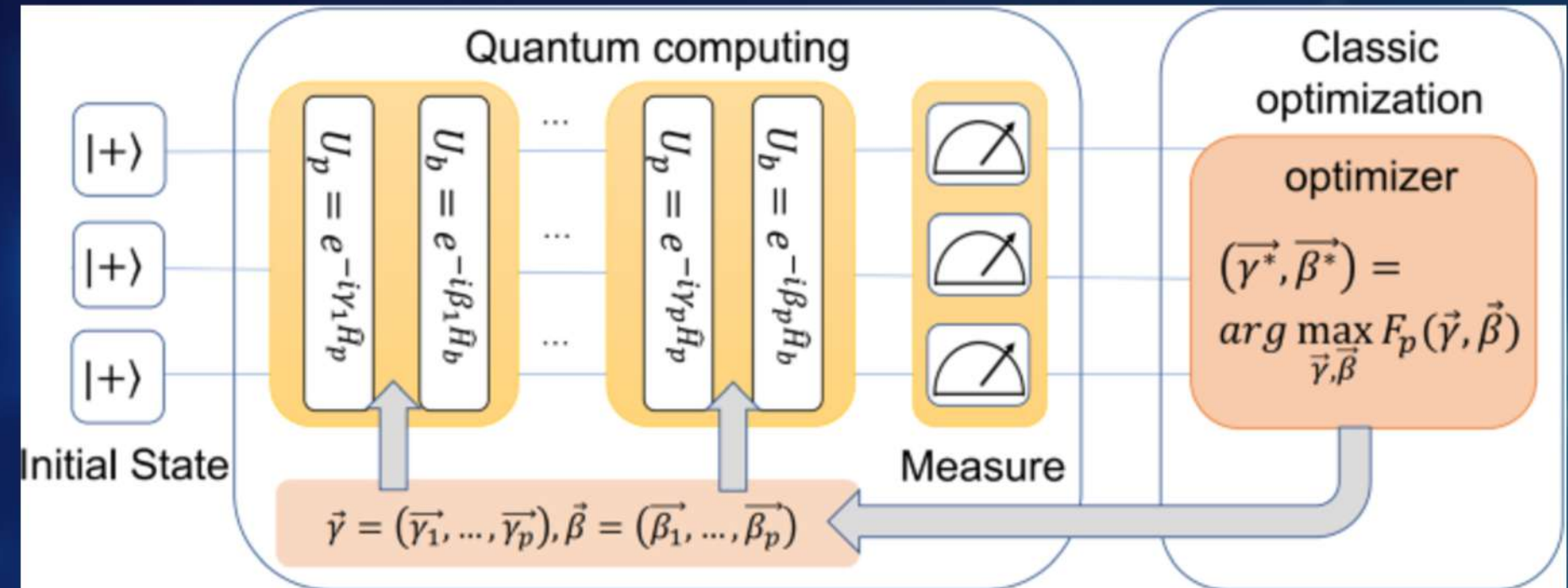
QAOA

Quantum Approximate Optimization Algorithm

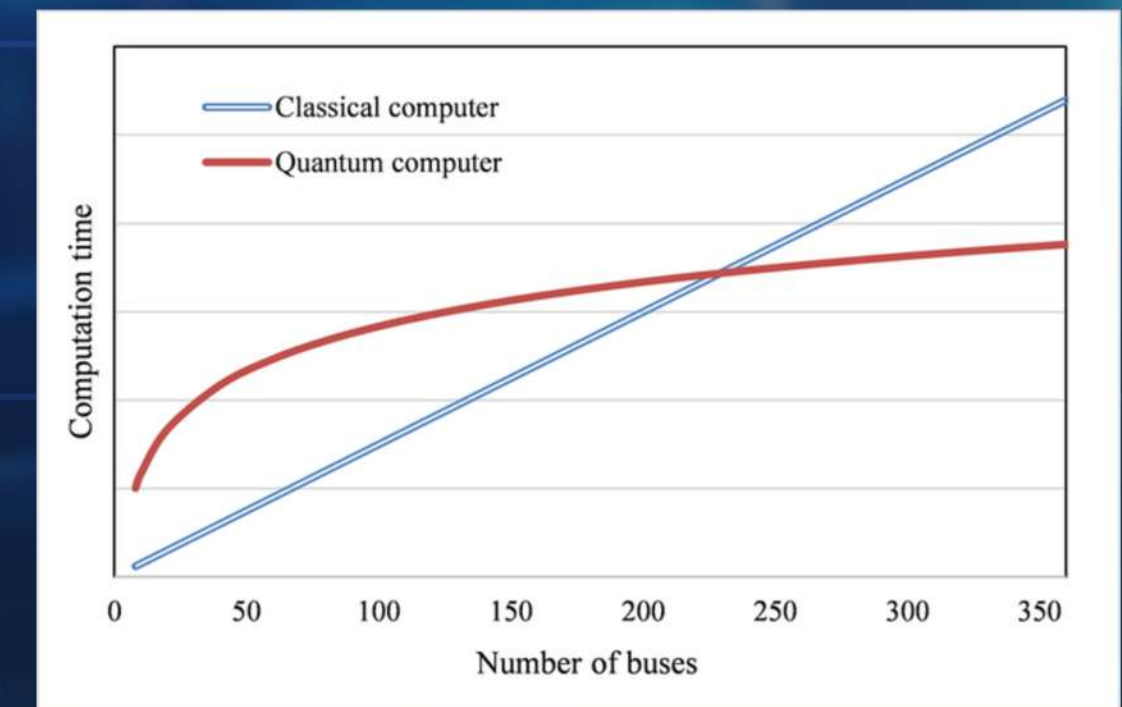




What Is QAOA



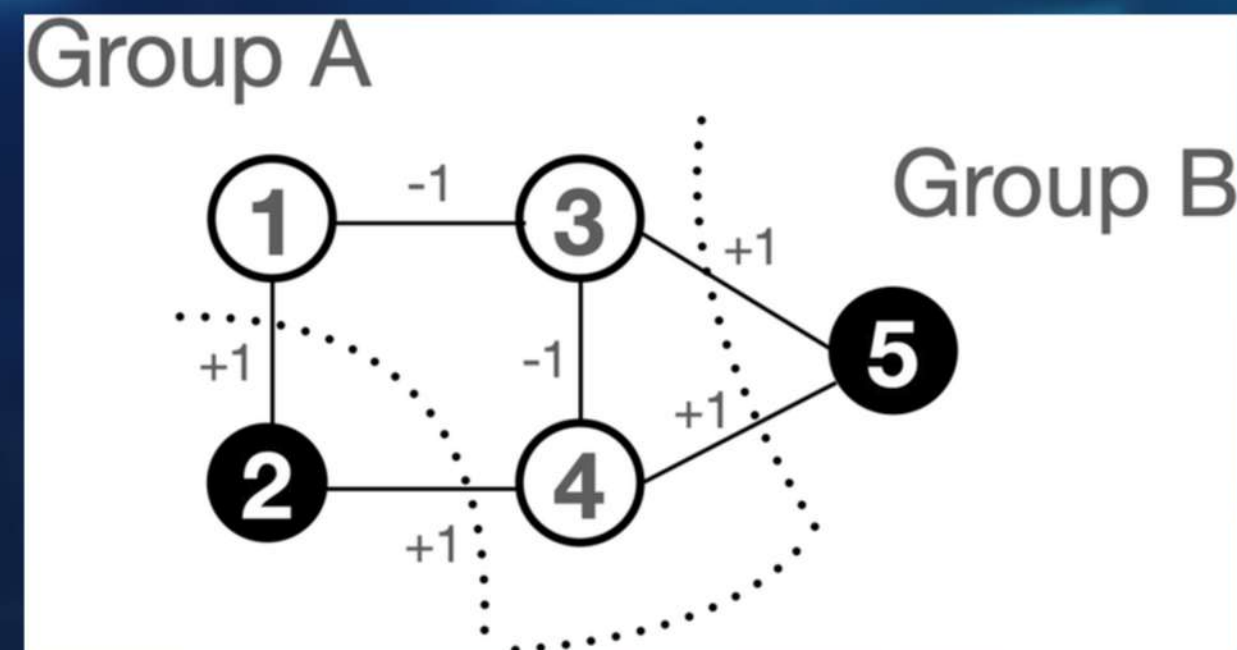
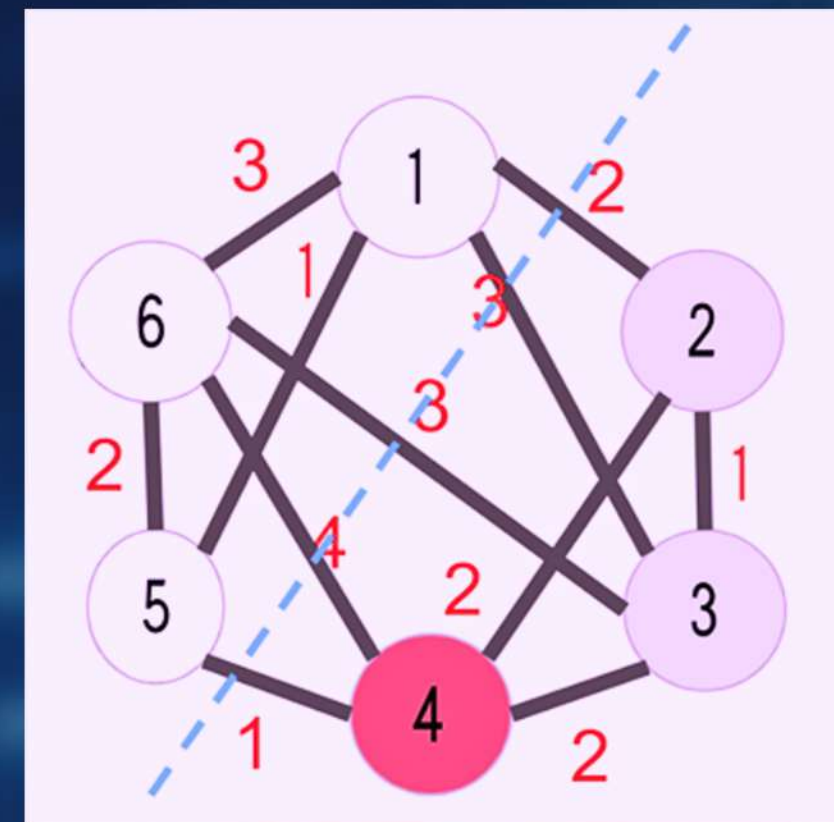
- The **Quantum Approximate Optimization Algorithm (QAOA)** is a hybrid-classical algorithm.
- It aims to solve combinatorial optimization problems by finding the optimal solution out of a large number of possibilities through a hybrid approach using classical and quantum computation methods.
- QAOA is specifically designed to work with today's noisy hardware due to its shallow circuit depth and hybrid nature.
- It is highly adaptable and can be used for a wide range of combinatorial optimization problems, like Max-Cut and the Traveling Salesman Problem.





Max Cut Problem

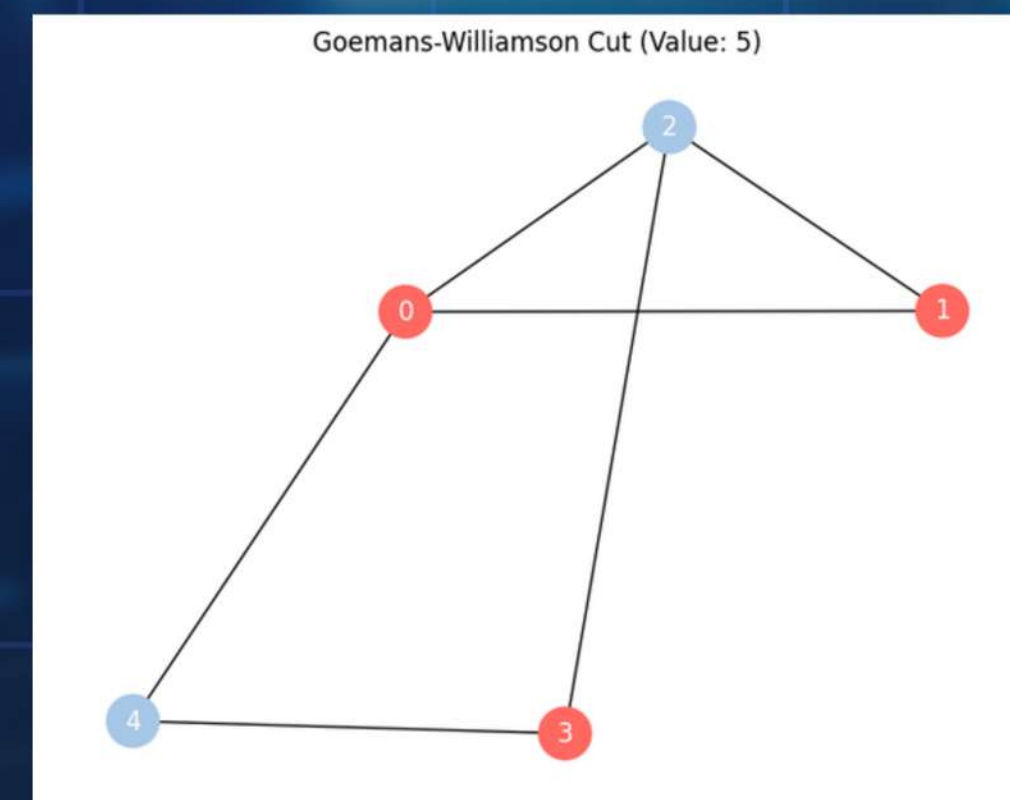
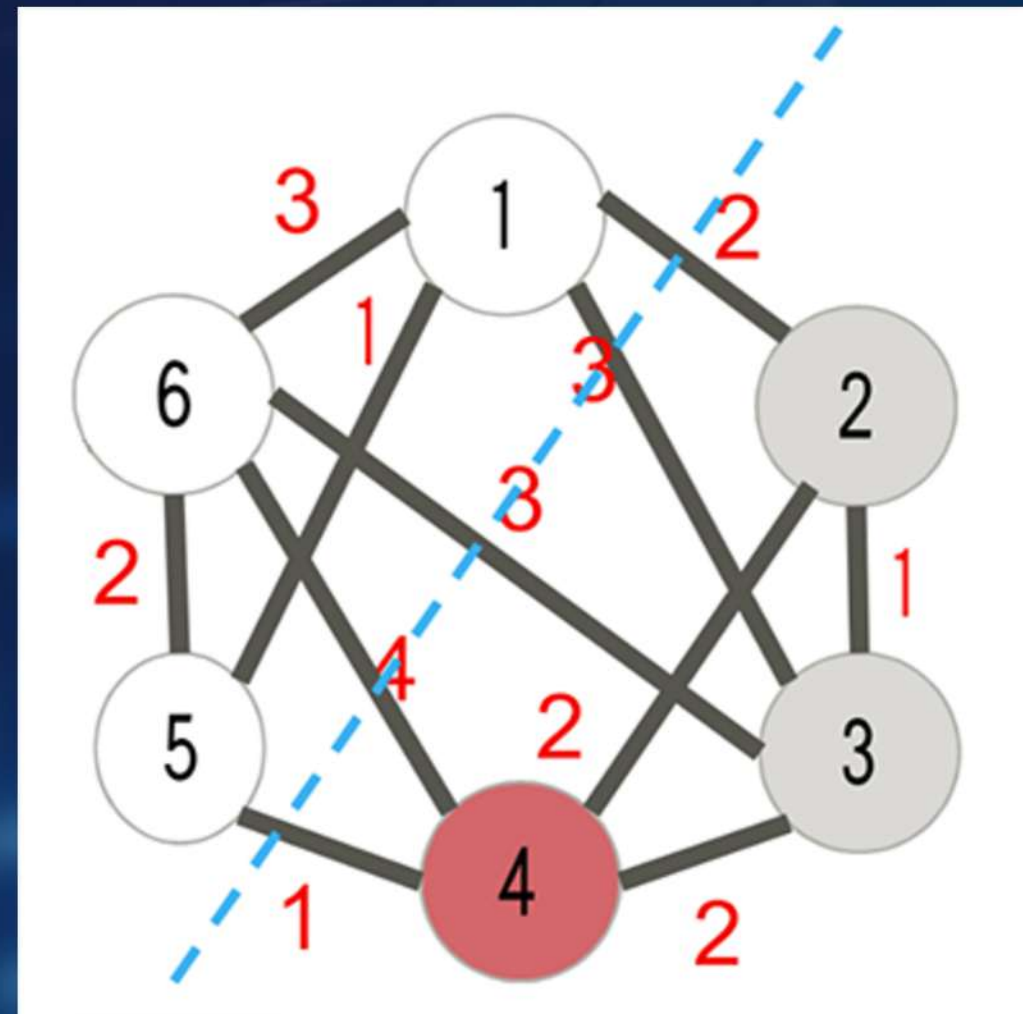
- Max-Cut is one of the classical NP-hard problems.
- Given a graph G with a certain number of nodes, the goal is to divide the nodes into two groups using a "cut" that maximizes the number of edges between these groups.
- **Analogy:** Splitting a group into two different teams for a game, where each edge represents rivalry. We aim to put rivals on opposite teams to make the game more fun and get the highest number of rivals possible between the teams.
- This problem is usually used to benchmark optimization problems, making it a perfect candidate for our experiment.





Goemans-Williamson

- The Goemans-Williamson algorithm is a well-known polynomial-time approximation algorithm for the Max-Cut problem.
- It is a classical method to solve the Max-Cut problem, utilizing the cvxpy library to solve the semidefinite programming (SDP) relaxation of the problem.
- This algorithm has an approximately 87.8% chance of finding the best cut.
- We used it to contrast our QAOA algorithm with a classical approach and test how well QAOA works.
- Example: For our 5-node graph, this algorithm gave a cut value of 5.





Why we chose QAOA?

- QAOA is specifically designed to work with today's noisy hardware to some level. Its shallow circuit depth and hybrid nature make it highly adaptable.
- The Max-Cut problem is uniquely adaptable to QAOA because it is represented using a cost Hamiltonian.
- This cost Hamiltonian is a mathematical operator that describes the total energy of a system, encoding the edges of the input graph, like giving each cut a score.
- QAOA leverages quantum states to explore the solution space more efficiently and find the best solution by minimizing or maximizing the cuts.
- In theory, QAOA, as a hybrid approach, is expected to exceed the 87.8% approximation ratio of Goemans-Williamson.



How we built it

- Tools: We used Qiskit libraries, such as AerSimulator, within a Python-based framework in a Google Colab notebook. Graph definition was done using rustworkx and networkx libraries.
- Graph Definition: We applied QAOA to solve the Max-Cut problem on small, unweighted graphs of 5, 10, and 20 nodes. Our initial implementation used a 5-node graph with edges: (0, 1), (0, 2), (0, 4), (1, 2), (2, 3), (3, 4).
- Cost Hamiltonian Construction: The Max-Cut cost Hamiltonian was constructed using the Pauli-Z operator. For each edge, a ZZ term was added with a unit weight, resulting in a sparse Pauli operator representing the input to the quantum circuit.
- QAOA Circuit Construction: The QAOA circuit was built using Qiskit's `qiskit.circuit.library` known as `QAOAAnsatz`. This class uses the cost Hamiltonian generated by the graph's adjacency matrix.
- Classical Optimization: For the optimization of QAOA, we used the COBYLA optimizer from `scipy.optimize.minimize`, which is a gradient-free method suitable for noisy optimization problems. For
- $p=1$, initial parameters were $[\pi/2, \pi/2]$.
- Visual: Consider showing a snippet of the code for `build_max_cut_paulis` or the `cost_hamiltonian`



How we ran it

- Simulator: All runs used Qiskit's AerSimulator.
- Noiseless: Baseline runs without noise models.
- Noisy: Used NoiseModel() with 2% depolarizing error on 2-qubit CX gates. Simulated NISQ hardware via fake backends.
- Circuit Depths: Tested depths $p = 1, 2, 3$; focused on $p = 1$ & 3 for noisy runs.
- Shots: 10,000 shots for 5 & 10-node graphs; reduced shots for 20-node due to Colab limits.
- Output: Saved as bitstring distributions.

```
# Run samplers
sampler = Sampler()
sampler_noisy = Sampler(run_options={"noise_model": noise_model})

job = sampler.run([optimized_circuit], shots=10000)
result = job.result()
quasi_dist = result.quasi_dists[0]

job_noisy = sampler_noisy.run([optimized_circuit_noisy], shots=10000)
result_noisy = job_noisy.result()
quasi_dist_noisy = result_noisy.quasi_dists[0]

[ ] # Convert to bitstrings
def to_bitstring(integer, num_bits):
    return [int(d) for d in np.binary_repr(integer, width=num_bits)]

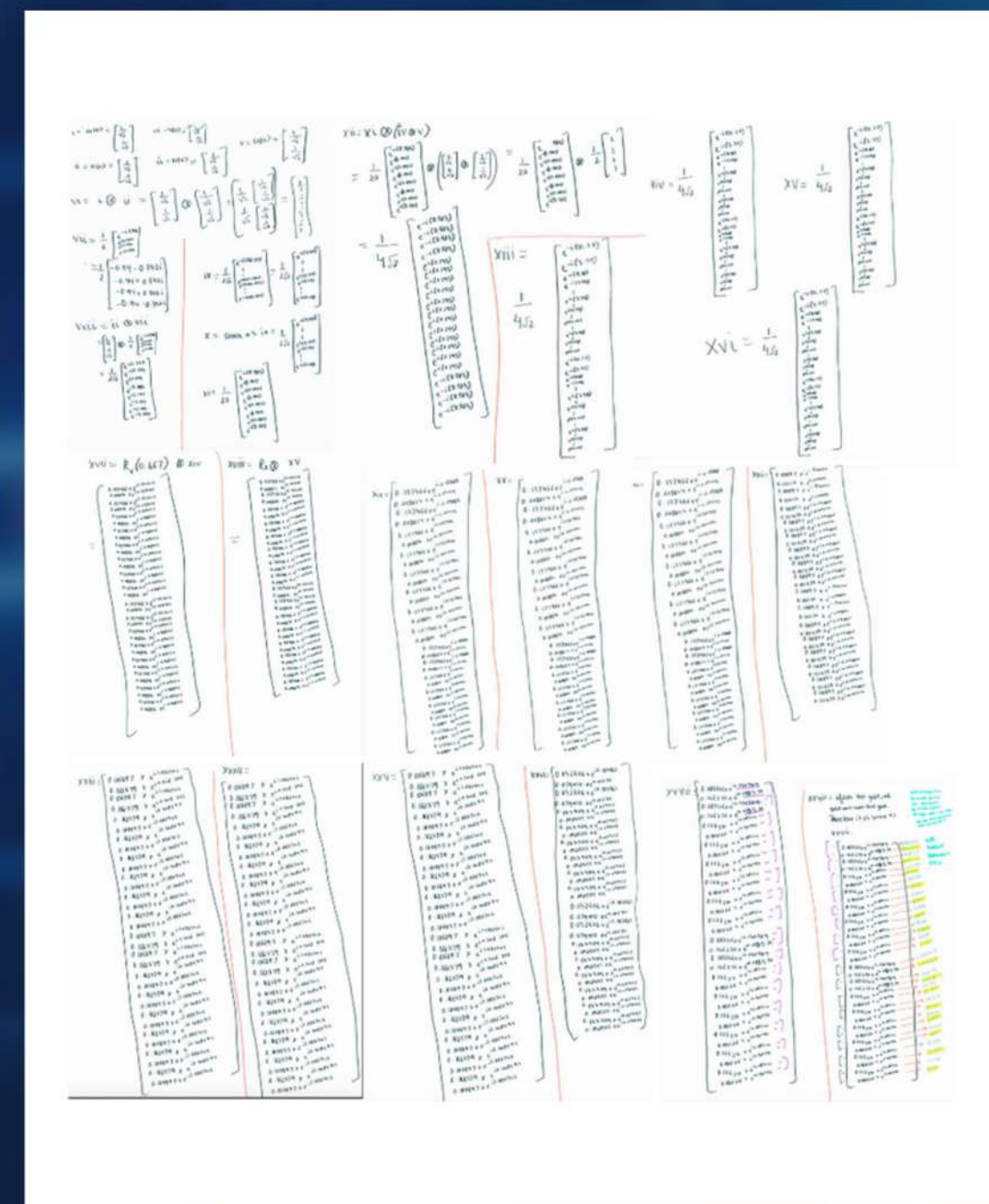
def process_distribution(dist):
    final_dist = {}
    for k, v in dist.items():
        if isinstance(k, str):
            final_dist[int(k, 2)] = float(v)
        else:
            final_dist[k] = float(v)
    return final_dist

final_dist = process_distribution(quasi_dist)
final_dist_noisy = process_distribution(quasi_dist_noisy)
```



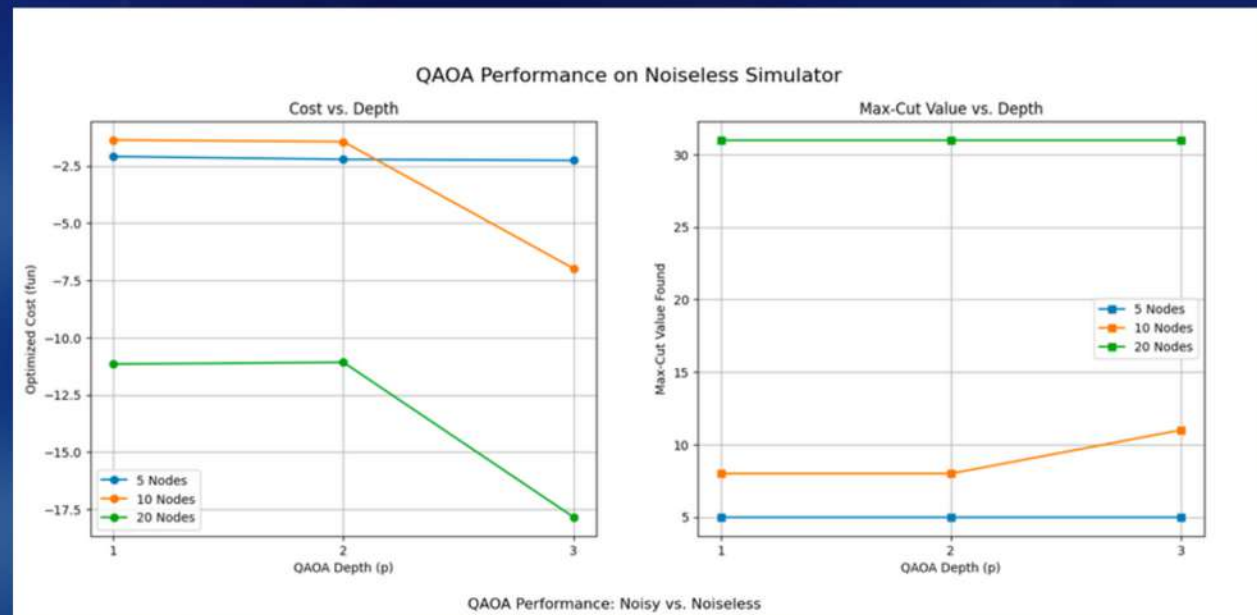

How We Traced It?

- We analyzed a 5-node QAOA circuit at depth $p = 1$, tracing each step in the circuit's evolution:
- 1. Initialization: All qubits start in the $|0\rangle$ state.
- 2. Superposition: Hadamard gates (H) create a uniform superposition—e.g., 32 equal-amplitude states for 5 qubits.
- 3. Cost Layer ($UC(\gamma)$): Applies $Z \otimes Z$ phase shifts based on the graph's edges—encoding the Max-Cut cost function.
- 4. Mixer Layer ($UM(\beta)$): Applies X-rotations to mix amplitudes, exploring new bitstrings.
- 5. Optimization & Measurement: γ and β are optimized classically. Measurement yields the bitstring (cut) with highest score.





What the data shows?



--- Noiseless Simulation Results ---

Optimized Cost Table (Noiseless):

	5 Nodes	10 Nodes	20 Nodes
p=1	-2.088	-1.369	-11.156
p=2	-2.215	-1.441	-11.082
p=3	-2.260	-6.984	-17.842

Max-Cut Value Table (Noiseless):

	5 Nodes	10 Nodes	20 Nodes
p=1	5	8	31
p=2	5	8	31
p=3	5	11	31

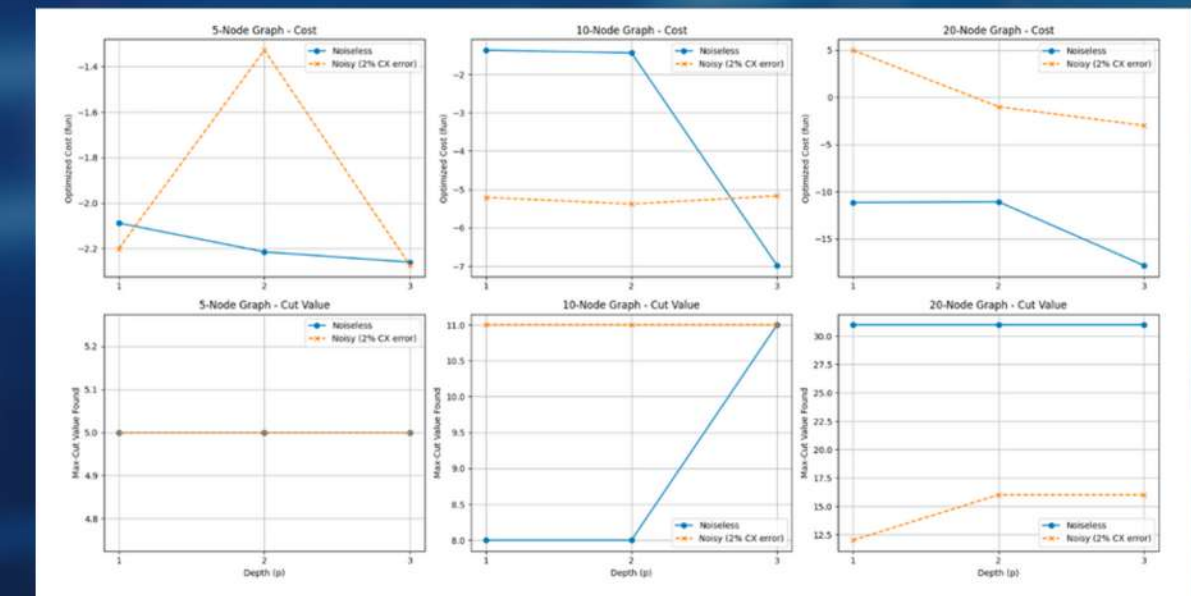
--- Noisy Simulation Results (2% CX Error) ---

Optimized Cost Table (Noisy):

	5 Nodes	10 Nodes	20 Nodes
p=1	-2.201	-5.207	5.0
p=2	-1.328	-5.379	-1.0
p=3	-2.275	-5.168	-3.0

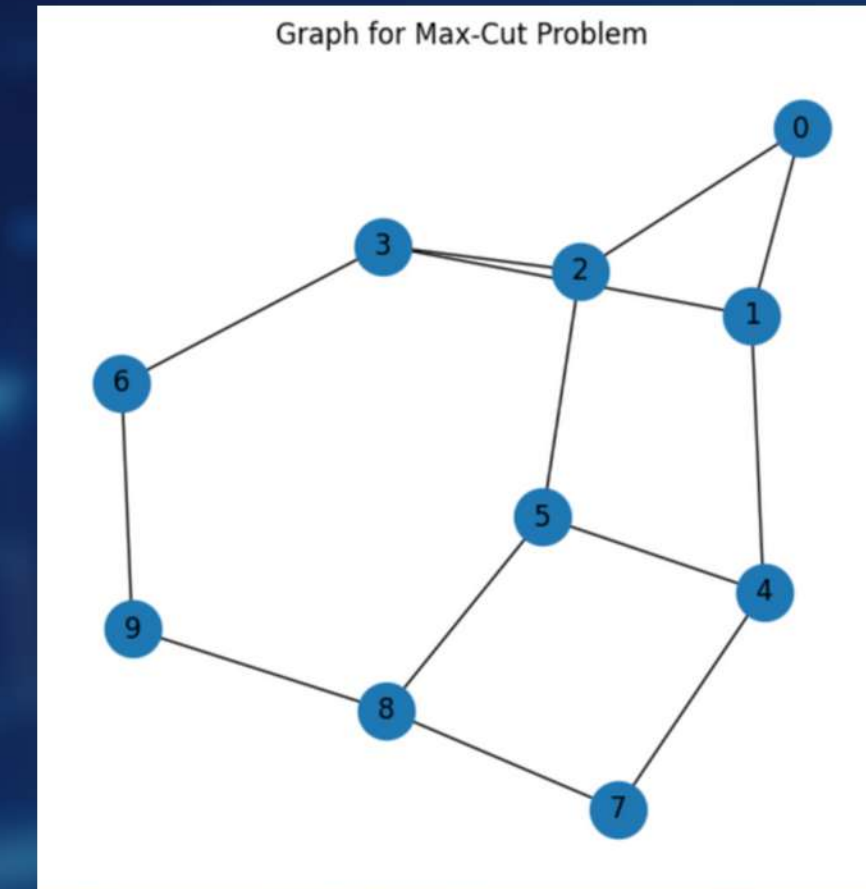
Max-Cut Value Table (Noisy):

	5 Nodes	10 Nodes	20 Nodes
p=1	5	11	12
p=2	5	11	16





What we changed



What we changed:

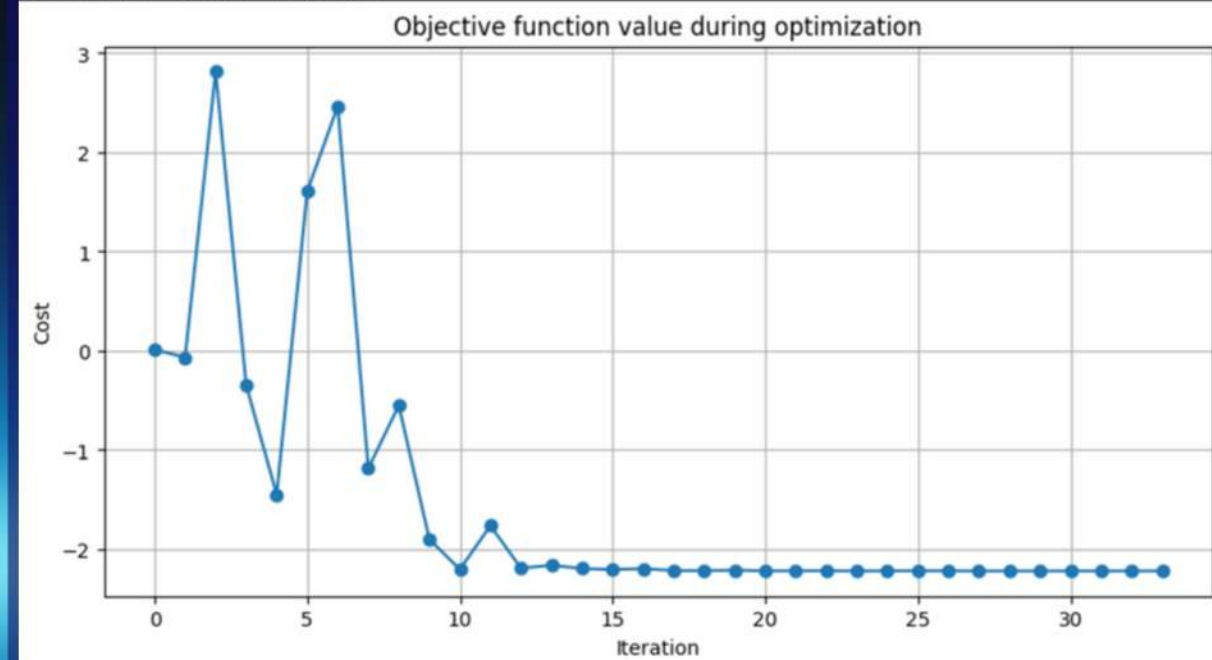
1. We scaled up the graphs while testing the algorithm
2. Adjusted the quantum depth
3. Removed Hadamard Gates



How it effects results?

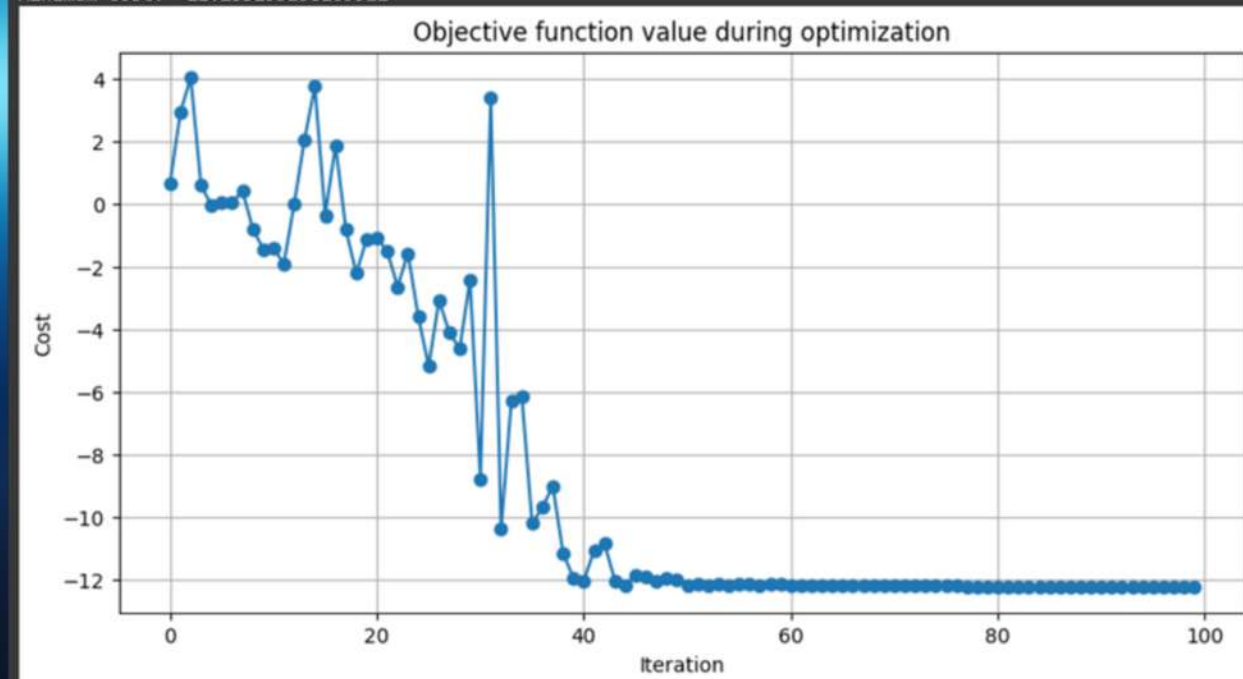
Quantum Depth = 1, Nodes = 5

Optimization completed.
Optimal parameters: [2.78661122 0.32451982]
Minimum cost: -2.2201091339199834



Quantum Depth = 3, Nodes = 20

Optimization completed.
Optimal parameters: [4.84407593 4.31864572 7.4582562 1.70308049 3.03430996 4.94825]
Minimum cost: -12.253293198169922



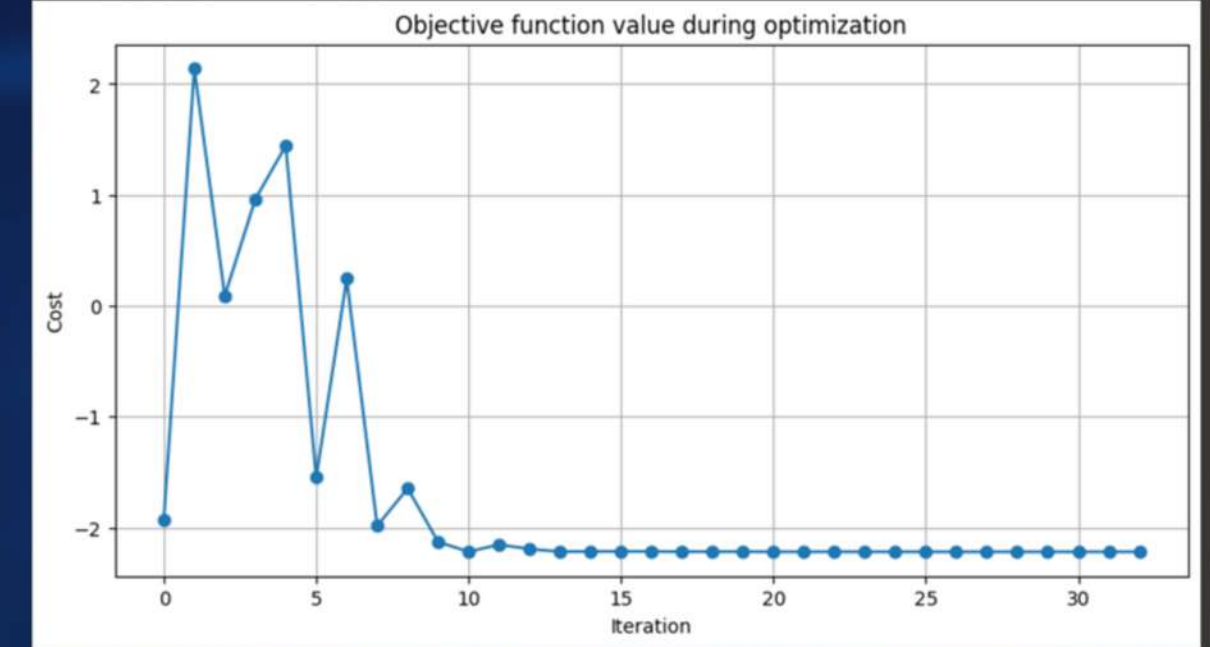
Noise

VS

No Noise

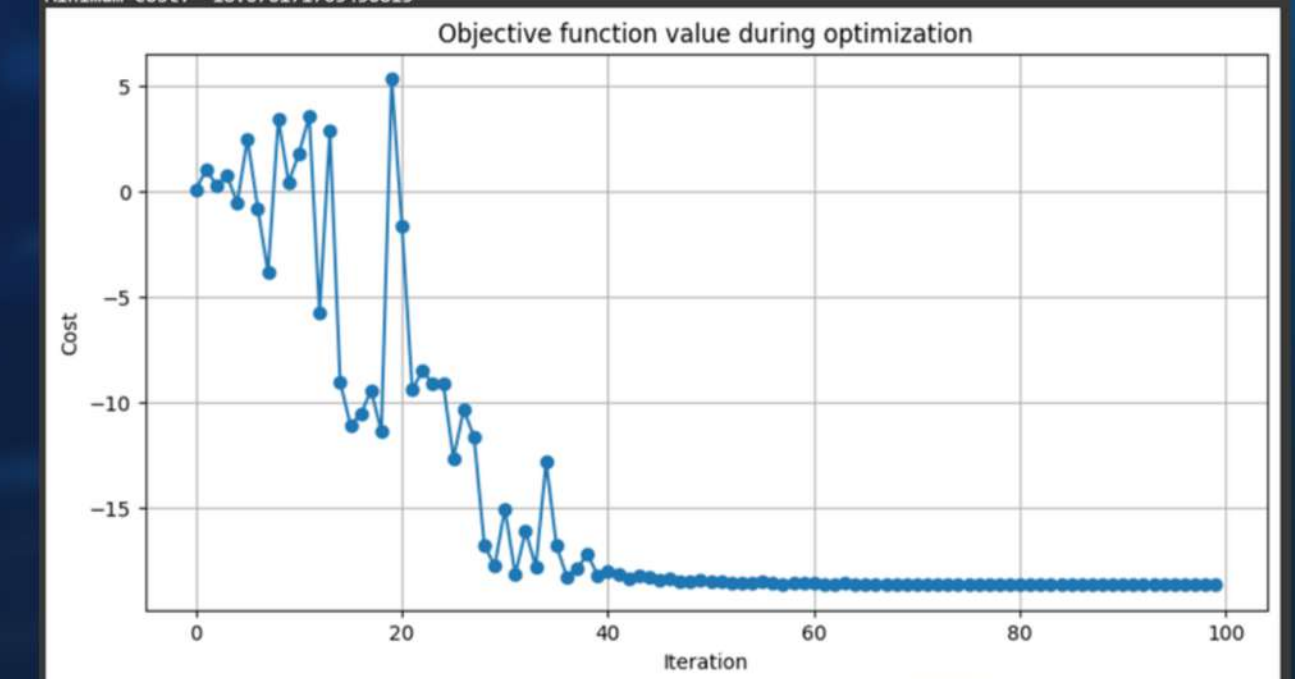
Quantum Depth = 1, Nodes = 5

Optimization completed.
Optimal parameters: [5.06675265 5.95967584]
Minimum cost: -2.220113697367405



Quantum Depth = 3, Nodes = 20

Optimization completed.
Optimal parameters: [2.98330829 5.80007923 5.55992869 3.39242928 2.59771345 3.27185383]
Minimum cost: -18.676171705436815





What we learned?

- Successfully ran QAOA for Max-Cut on 5, 10, and 20-node graphs using Qiskit.
- Circuit Depth: In noiseless runs, deeper circuits ($p=1-3$) yielded better results.
- Noise Impact: QAOA performance drops with noise, especially on larger graphs.
- Classical Comparison: QAOA can rival Goemans-Williamson in ideal settings.
- Real-World Limits: With noise, classical methods still outperform QAOA.
- Smaller Graphs: QAOA performs well under noise for small graphs.
- Key Insight: Noise significantly affects outcomes error mitigation is critical.



Future ideas

- Error Mitigation: Invest in noise reduction strategies to improve QAOA reliability.
- Alternative Optimizers: Try optimizers like SPSA for better performance under noise.
- Scaling Up: Test QAOA on larger, more complex graphs as hardware improves.
- Beyond Max-Cut: Apply QAOA to other problems like the Traveling Salesman Problem.